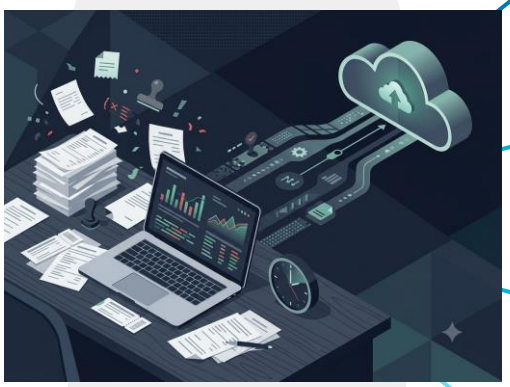


Cloud-Based Algorithmic Trading & Portfolio Management System

K. Vishal Kumar
RA2311003011187
T-1

Problem statement & motivation



01 Manual trading requires continuous monitoring of market conditions and portfolio positions.

Manual Monitoring

02 Locally running trading scripts depend on the availability of a personal computer and a stable connection.

Local

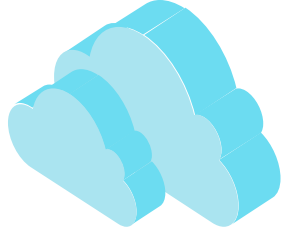
03 Disconnected market data, strategy execution, and portfolio records can lead to inconsistent decisions.

Reliability & Risk

04 Repeated investments and portfolio rebalancing require systematic execution and risk controls.

Portfolio Tracking

Objectives and System Requirements



LEFT SECTION

Traditional Challenges

Manual Market Monitoring
Continuous tracking of prices and portfolio positions

Local Execution Dependency
Scripts depend on a personal computer and stable connectivity

Disconnected Records
Market data, strategy signals and portfolio records are fragmented

Inconsistent Risk Controls
Repeated investments and rebalancing need systematic validation

MIDDLE SECTION



Cloud-Hosted Paper Trading

Market Data & Indicators

Rule-Based BUY / SELL / HOLD

Portfolio & Order Tracking

RIGHT SECTION

Outcome

Remote Accessibility
Access the dashboard through a browser

Structured Portfolio Records
Persist simulated trades and holdings

Validated Paper Orders
Apply cash, holdings and duplicate-signal checks



Proposed solution & system workflow

Cloud-hosted paper-trading dashboard. Users review market data and signals, then trigger a strategy trade or submit a manual paper order. Orders are validated and portfolio updates are stored.

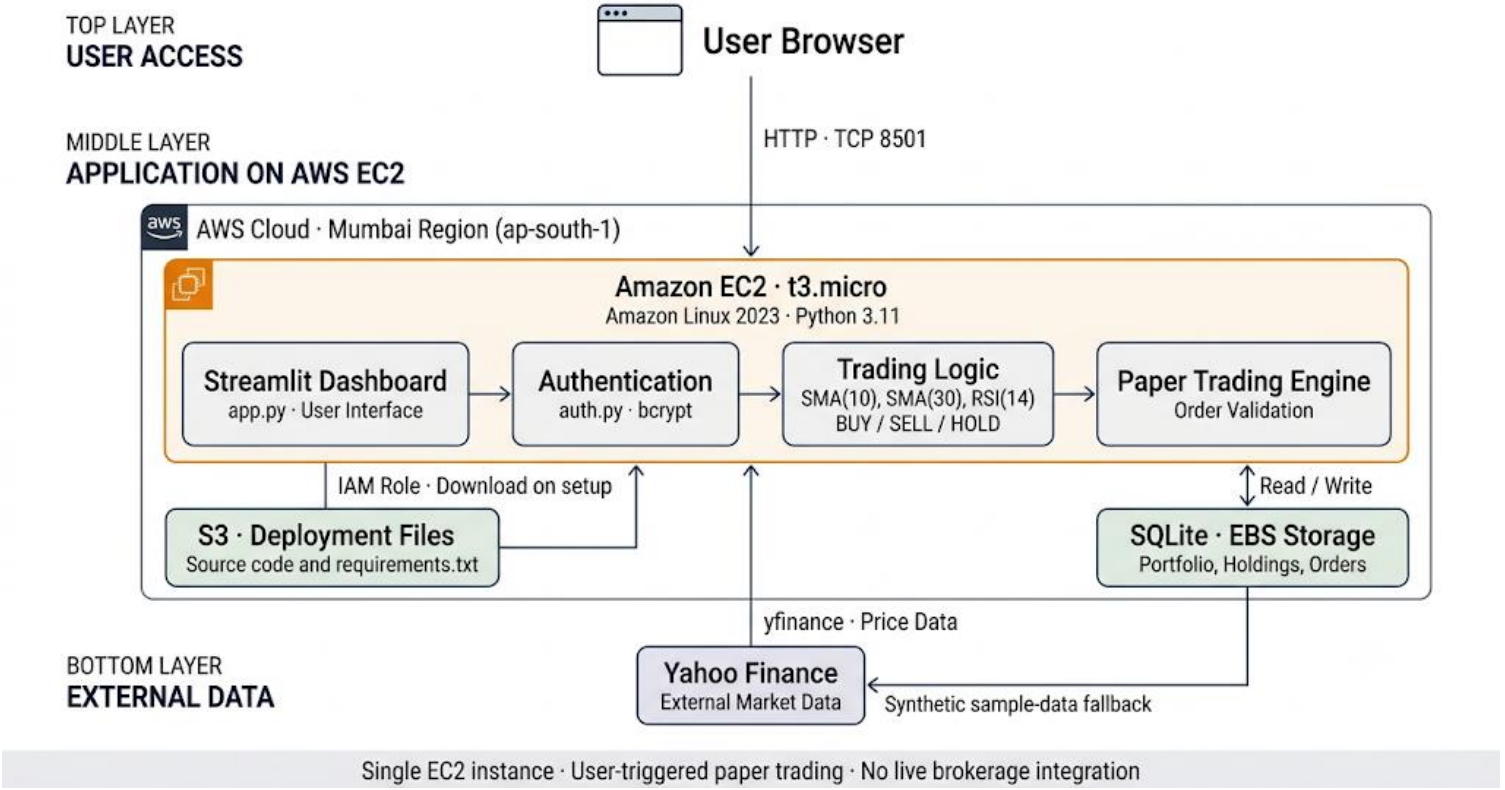
Browser	Login	Market data	SMA + RSI	Trade engine	SQLite
Open dashboard	Authenticate	Fetch or sample	Generate signal	Validate order	Persist result

Paper trading only — no broker integration or real-money transactions.



Implementation & technical execution

Cloud architecture:



IaaS, PaaS & SaaS selection
Cloud Service Models: Selection and Justification

IaaS

AWS EC2



Compute environment with control over the application runtime

PaaS / managed service

SQLite + EBS



SQLite file on the EC2 instance's EBS-backed filesystem

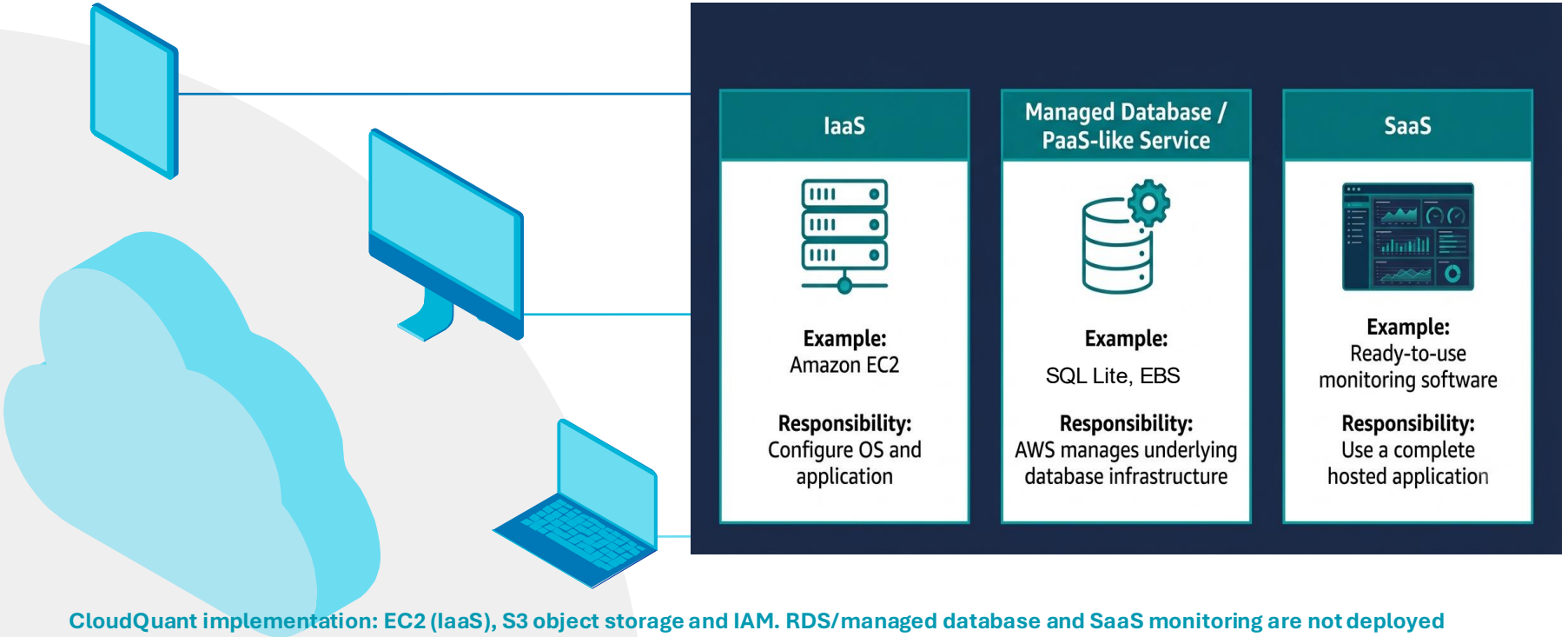
SaaS

External data service: Yahoo Finance via yfinance



External market data; not an AWS monitoring service

IaaS, PaaS & SaaS selection



CloudQuant implementation: EC2 (IaaS), S3 object storage and IAM. RDS/managed database and SaaS monitoring are not deployed components.

Technology Stack & Database Design

Actual implementation — SQLite

Application stack

- Python + Streamlit
- Pandas / NumPy
- yfinance market data
- SMA(10), SMA(30), RSI(14)
- bcrypt authentication









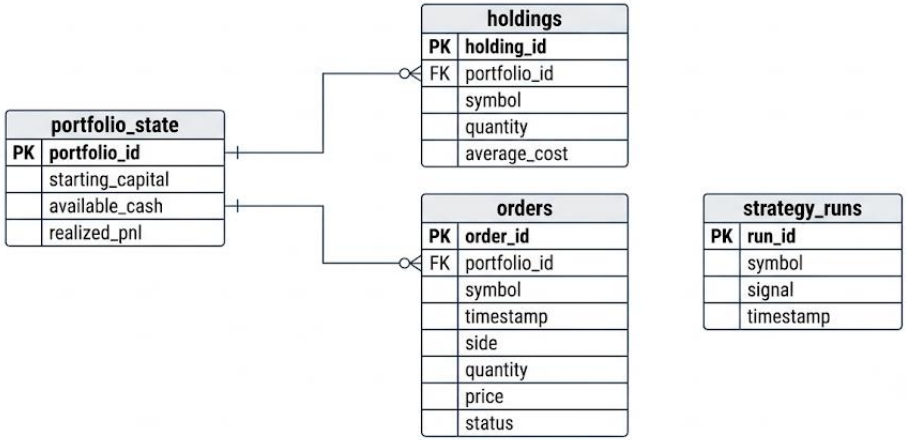




SQLite tables

CloudQuant — SQLite Database Schema

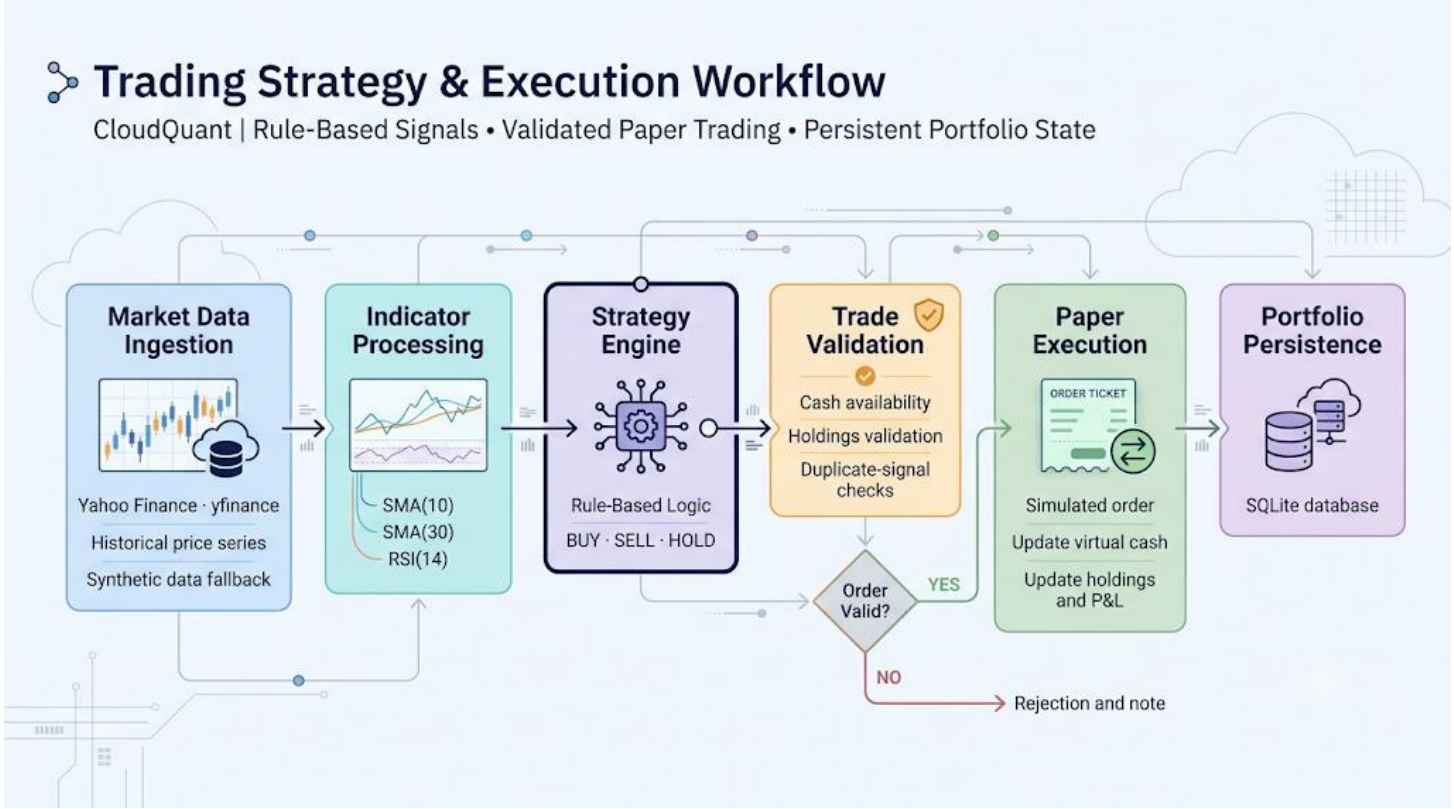
Persistent state for simulated portfolio management and paper trading



PK = Primary Key | FK = Foreign Key

Cloud services: EC2 (compute) • EBS (database file storage) • S3 (deployment artifacts) • IAM (permissions)

Trading strategy & execution workflow:



No scheduled trading job and no live brokerage execution.

Cloud deployment & configuration



Provision an EC2 instance and configure its security group.

EC2

Python

Install Python and the application's dependencies.

Bootstrap the EC2 instance; download app files from S3 using the attached IAM role.

SQLite3
SQL database engine

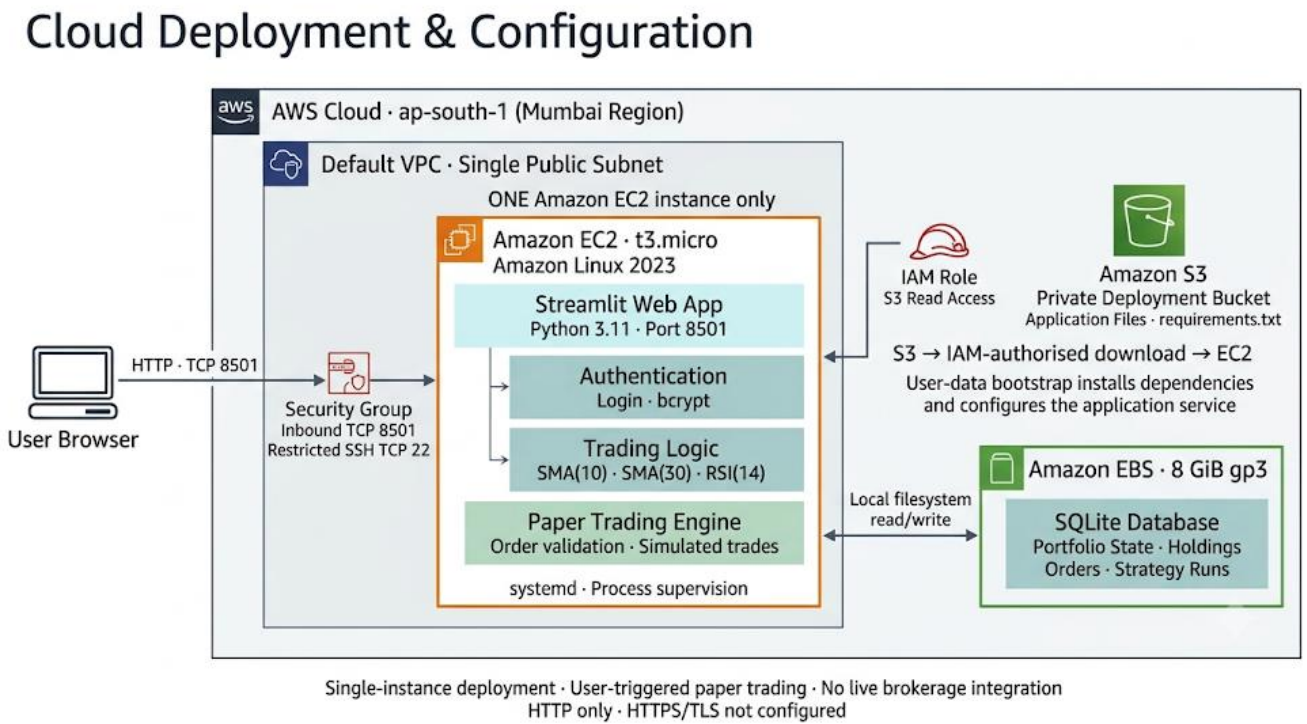
Store application state in SQLite on the EC2 instance's EBS-backed filesystem.

Configure the app for user-triggered strategy execution and manual paper orders.

Enable the systemd service and verify the Streamlit endpoint and service logs.

Cloud deployment & configuration

Actual deployment: one EC2 instance in one public subnet / Availability Zone



HTTP only • single instance • no NAT Gateway, private database subnet or Multi-AZ deployment

Networking, security & reliability:

01 Network security

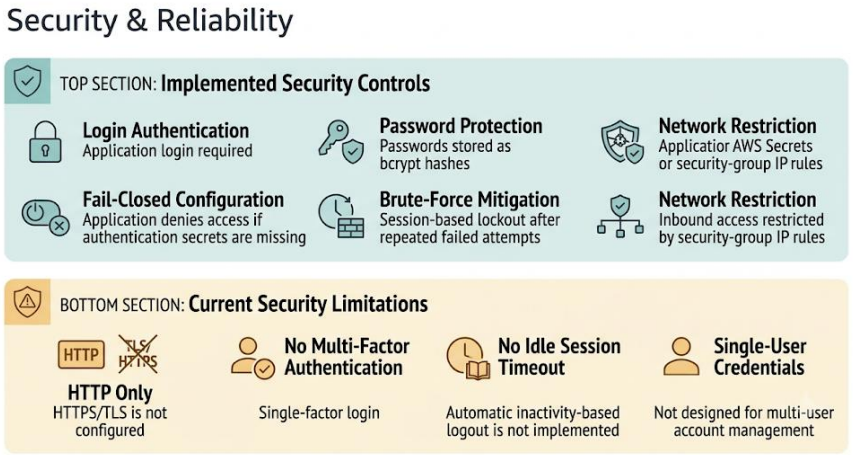
- Restrict inbound traffic using AWS security groups.
  Expose only the required application ports.
  Limit database access to the application server.
  Current deployment uses HTTP; HTTPS/TLS is future work.

02 Identity and credentials

- Apply least-privilege IAM permissions.
  Keep database credentials and API keys outside source code.
  Credentials stored in .streamlit/secrets.toml with restrictive file permissions; not AWS Secrets Manager.

03 Reliability controls

- Block duplicate strategy signals under defined conditions.
  Market data can fall back to synthetic sample data.
  Use database transactions for portfolio updates.
  systemd and application logs aid troubleshooting; CloudWatch is not configured.



Performance & Technical Evaluation

Time

Measure page-load and user-triggered strategy execution time under defined test conditions.

Portfolio & Database

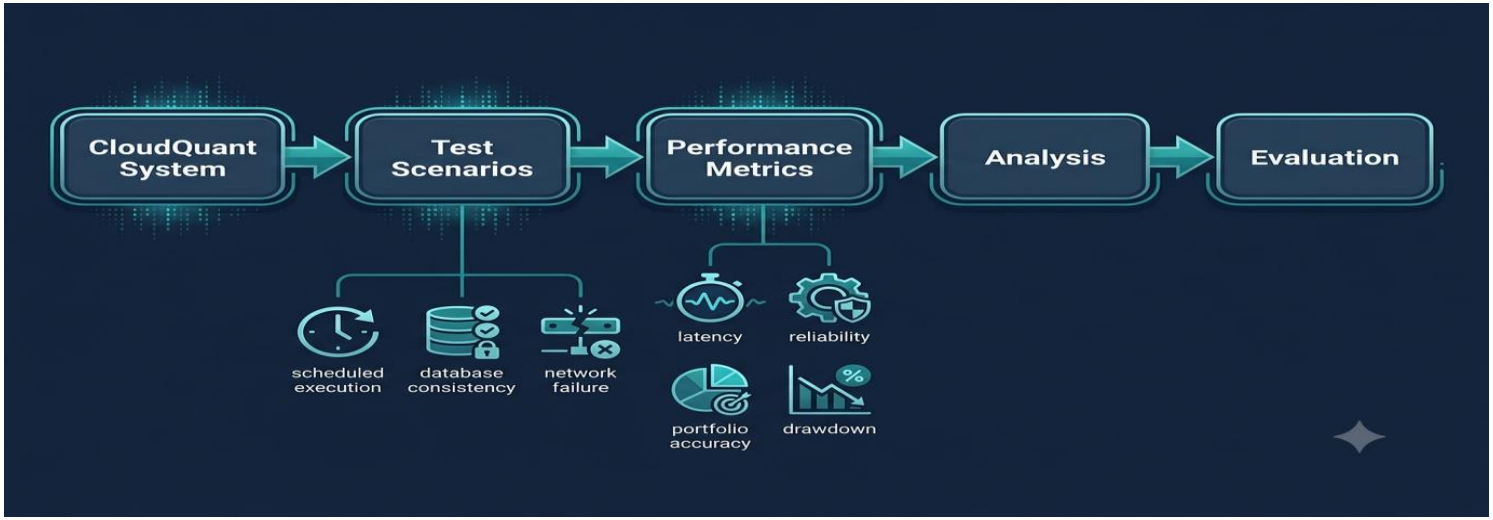
Assess portfolio update accuracy and database consistency.

System Reliability

Test external-data failure and confirm the synthetic-data fallback behaves as intended.

Historical Performance

Backtest strategy returns and risk only after a documented backtesting procedure is implemented.



Performance & Technical Evaluation:

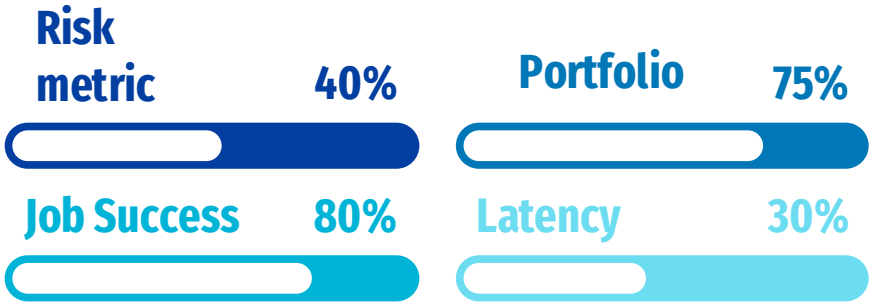
Report only measured results; no unsupported percentage claims

30 unit tests reported passing

Authentication + trading-engine tests

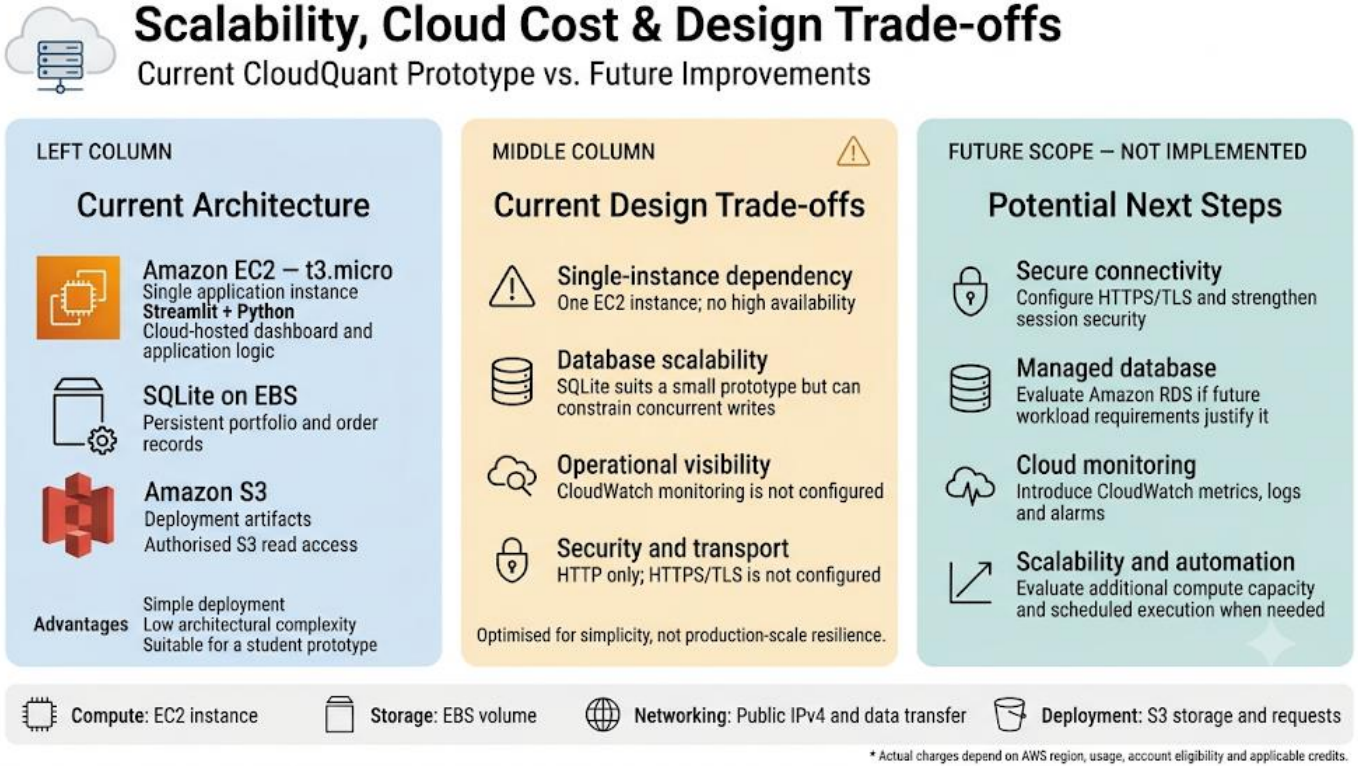
Functional checks to demonstrate

- Login success / failure
- Insufficient-cash rejection
- Oversell rejection
- Duplicate signal protection



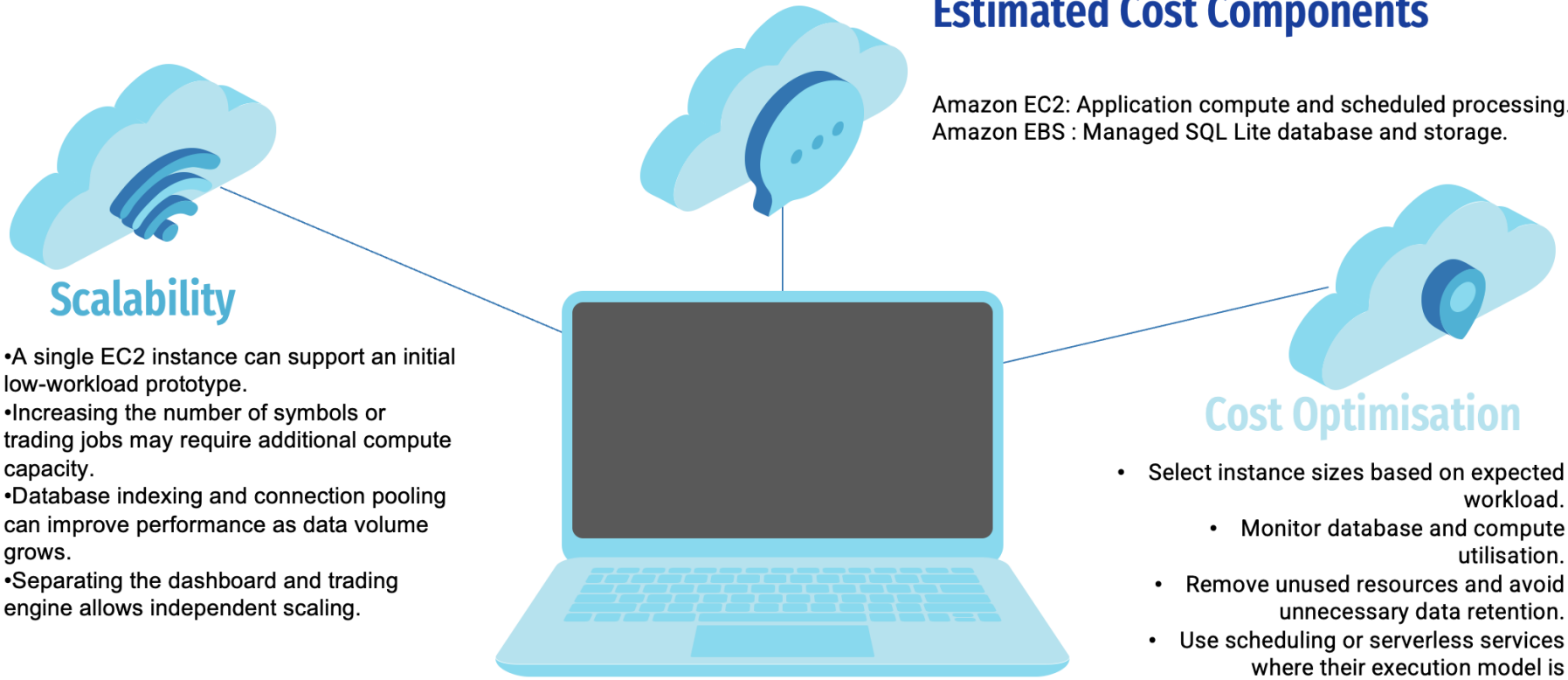
30 passing unit tests are not evidence of strategy profitability or production-scale performance.

Scalability, Cloud Cost and Design Trade-offs



Horizontal scaling is not a simple add-on while application state remains in a local SQLite file on one instance.

Scalability, Cloud Cost and Design Trade-offs



Estimated Cost Components

Amazon EC2: Application compute and scheduled processing.
Amazon EBS : Managed SQL Lite database and storage.

Scalability

•A single EC2 instance can support an initial low-workload prototype.
•Increasing the number of symbols or trading jobs may require additional compute capacity.
•Database indexing and connection pooling can improve performance as data volume grows.
•Separating the dashboard and trading engine allows independent scaling.

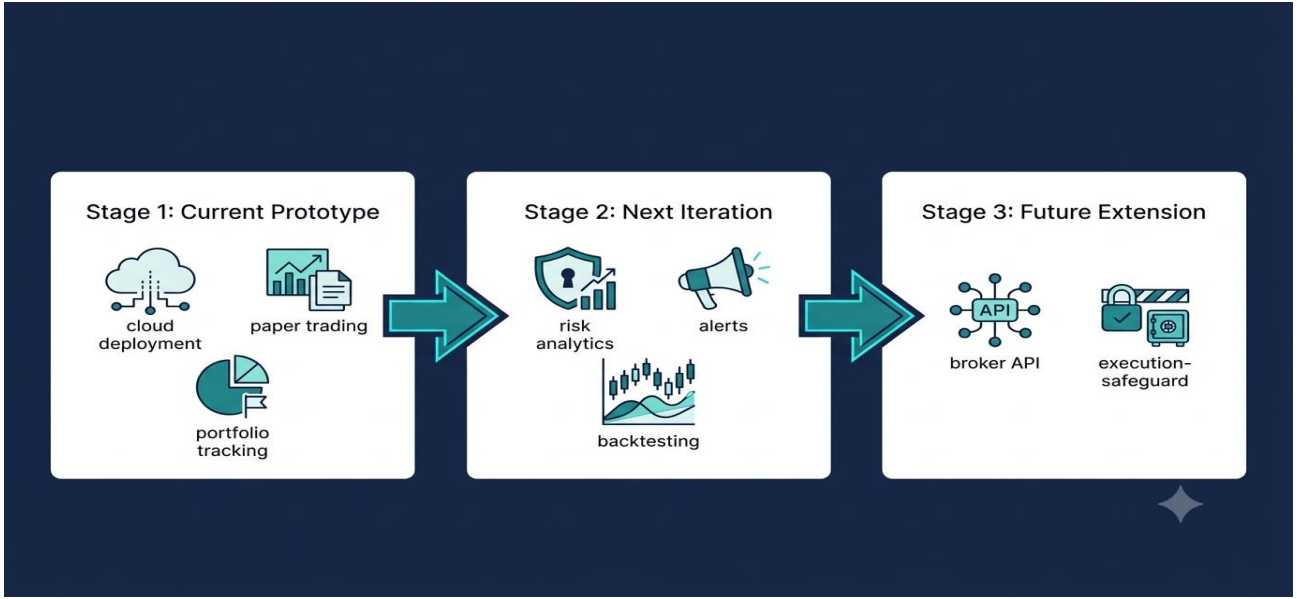
Cost Optimisation

- Select instance sizes based on expected workload.
- Monitor database and compute utilisation.
- Remove unused resources and avoid unnecessary data retention.
- Use scheduling or serverless services where their execution model is appropriate.

Conclusion & Future Scope

Conclusion:

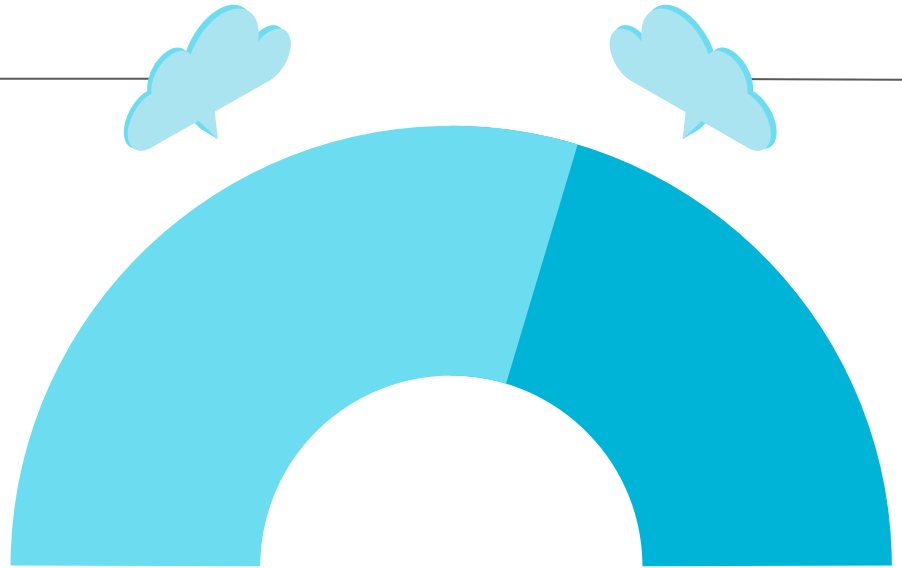
- CloudQuant is a cloud-hosted paper-trading dashboard deployed on AWS EC2.
- SMA/RSI signals and manual paper orders support rule-based experimentation.
- SQLite persists the portfolio and order history on EBS-backed storage.
- Authentication and order-validation checks improve access control and simulated ledger integrity.
- Strategy execution is user-triggered; scheduled trading and CloudWatch monitoring are not implemented.



Conclusion & Future Scope

Limitations

- Cloud resources may incur ongoing charges; actual cost depends on usage and account offers.



Future Scope

- Broker API integration with appropriate safeguards.
- More advanced portfolio optimisation and risk analytics.
- Event-driven processing and automated alerts.
- Improved backtesting and workload-based scaling.